

Jurnal Praktikum
Sistem Operasi
Modul 6: Proses Sekuensial dan Konkuren

Nama : Aulia Ahmad Ghaus Adzam NIM : 2311104028 Kelas : SE-07-01

Tujuan:

1. Mahasiswa mampu membuat proses berjalan secara sekuensial.
2. Mahasiswa mampu membuat proses berjalan secara konkuren.

Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir.
2. Untuk soal source code, kumpulkan SS-nya saja.
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username.
4. Berikan identitas nama - nim dalam bentuk comment di Source Code.
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

Jurnal

1. **[10 Poin]** Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

Jawab = Maksimal proses 8

```
#ifndef NPROC
#define NPROC      8
#endif
```

Ubah menjadi 150 =

```
#ifndef NPROC
#define NPROC      150
#endif
```

2. **[20 Poin]** Jalankan kode sekuensial!

Jawab =

Source Code

```
/* main.c - main */
#include <xinu.h>

void A(){
    kprintf("Proses A jalan\n");
}
void B(){
    kprintf("Proses B jalan\n");
}

process main (void)
{
    kprintf("=== SEKUESIAL ===\n");
    A();
    B();
    return OK;
}
```

Output Code =

```
Obtained IP address 10.0.3.15 (0x0a00030f)
=== SEKUESIAL ===
Proses A jalan
Proses B jalan
```

3. **[20 Poin]** Jalankan kode konkuren!

Jawab =

Source Code =

```

process A(){
    kprintf("Proses A jalan\n");
    return OK;
}
process B(){
    kprintf("Proses B jalan\n");
    return OK;
}
process main (void)
{
    kprintf("=== Konkuren ===\n");
    resume(create(A, 1024, 20, "A", 0));
    resume(create(B, 1024, 20, "B", 0));
    sleepms(500);
    return OK;
}

```

Output Code =

```

Obtained IP address 10.0.3.15 (0x0a00030f)
=== Konkuren ===
Proses A jalan
Proses B jalan

```

4. **[50 Poin]** Buatlah 2 **proses** produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```

int32 n = 0; //global variabel

void produser(void){ int32
    i;
    for (i=1; i<=1000; i++){ n++;
    }
}

void konsumer(void){ int32
    i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n); }
}

```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal). Jelaskan mengapa hasilnya seperti itu!

Jawab =

Output program dapat menjadi tidak konsisten karena proses produser dan consumer dieksekusi secara bersamaan serta mengakses variable global n tanpa mekanisme sinkronisasi yang memadai. Kondisi ini memungkinkan consumer memperoleh nilai n pada waktu yang tidak terprediksi, misalnya sebelum, Ketika, atau setelah produser melakukan perubahan terhadap variable tersebut. Situasi ini dikenal sebagai race condition, yaitu Ketika hasil eksekusi program bergantung pada urutan akses beberapa proses atau thread terhadap sumber daya yang sama.